REȚELE DE CALCULATOARE

Săptămâna 6

Nivelul transport: TCP (Transmission Control Protocol), UDP (User Datagram Protocol), porturi, transfer fiabil și congestie

Curs • Seminar • Laborator (variantele Windows și Linux)

Cuprins

Obiective de învățare și locul în cadrul disciplinei

Partea I — Curs

1. Rolul și serviciile nivelului transport
2. Protocoale de transport
3. Numere de port, socket-uri și multiplexare
4. UDP (User Datagram Protocol)
5. TCP (Transmission Control Protocol)
6. Mecanisme TCP (Transmission Control Protocol): stabilirea conexiunii în trei pași, transmisia stop-and-wait, fereastra glisantă
7. Congestia traficului și controlul congestiei în TCP (Transmission Control Protocol)
8. Rezumat și termeni-cheie

Partea a II-a — Seminar: probleme rezolvate și exerciții

Partea a III-a — Laborator

Laboratorul 6-W: nivelul transport în Windows

Laboratorul 6-L: nivelul transport în Linux

Model de referat de laborator și bibliografie

Obiective de învățare și locul în cadrul disciplinei

Săptămânile 6 și 7 acoperă împreună nivelul transport și interfața sa de programare. **Săptămâna aceasta (săptămâna 6)** se concentrează pe nivelul transport propriu-zis: ce servicii oferă, prin ce diferă UDP (User Datagram Protocol) și TCP (Transmission Control Protocol), cum identifică numerele de port procesele, cum deschide și închide TCP (Transmission Control Protocol) conexiunile, cum livrează datele în mod fiabil (transmisie stop-and-wait și cu fereastră glisantă) și cum reacționează la congestia traficului. **Săptămâna viitoare (săptămâna 7)** construiește pe această bază, cu socket-uri și programare client-server în Java peste TCP (Transmission Control Protocol) și UDP (User Datagram Protocol).

Tabelul 1. *Cum este împărțit capitolul despre nivelul transport între săptămânile 6 și 7*

Săptămâna	Teme de curs	Seminar	Laborator
6 (săptămâna aceasta)	Protocoale de transport; UDP (User Datagram Protocol); TCP (Transmission Control Protocol); numere de port; stabilirea conexiunii în trei pași; stop-and-wait; fereastră glisantă; congestie	Identificarea porturilor/socket-urilor, eficiența stop-and-wait comparativ cu fereastra glisantă, urmărirea numerelor de secvență/ACK (Acknowledgment – confirmare), evoluția ferestrei de congestie, decodificarea antetelor	Inspectarea porturilor și conexiunilor, capturarea stabilirii conexiunii, trafic UDP (User Datagram Protocol) comparativ cu TCP (Transmission Control Protocol), setări TCP (Transmission Control Protocol) și controlul congestiei (Windows și Linux)
7 (săptămâna viitoare)	Socket-uri: definiții, clasificare, API (Application Programming Interface – interfață de programare a aplicațiilor)-ul socket; programare client-server TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) în Java	Secvențe de apeluri socket, delimitarea mesajelor, depanarea codului cu socket-uri, proiectarea protocoalelor	Realizarea și testarea clienților și serverelor Java TCP (Transmission Control Protocol)/UDP (User Datagram Protocol) (Windows și Linux)

La finalul acestei săptămâni, studenții ar trebui să poată:

1. Să explice rolul nivelului transport și diferența dintre livrarea de la proces la proces și cea de la gazdă la gazdă.
2. Să compare TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) și să aleagă protocolul potrivit pentru o aplicație dată.
3. Să folosească numerele de port și 5-tuplul pentru a explica multiplexarea și demultiplexarea; să încadreze porturile în intervalele binecunoscute, înregistrate și dinamice.
4. Să descrie antetele UDP (User Datagram Protocol) și TCP (Transmission Control Protocol) și să le interpreteze câmpurile într-o captură de pachete.
5. Să urmărească stabilirea (în trei pași) și închiderea unei conexiuni TCP (Transmission Control Protocol), inclusiv numerele de secvență și de confirmare.
6. Să calculeze eficiența transmisiei stop-and-wait și a celei cu fereastră glisantă și să dimensioneze o fereastră pe baza produsului lățime de bandă × întârziere.
7. Să explice cauzele congestiei și algoritmii TCP (Transmission Control Protocol) de control al congestiei (pornire lentă, evitarea congestiei, retransmisie rapidă/recuperare rapidă).
8. Să folosească instrumentele sistemului de operare (netstat/ss, PowerShell, tcpdump, Wireshark) pentru a observa comportamentul nivelului transport în Windows și Linux.

Partea I — Curs

1. Rolul și serviciile nivelului transport

Nivelul rețea (IP (Internet Protocol)) livrează pachete **de la o gazdă la alta**, după principiul „best effort”: pachetele se pot pierde, se pot duplica, pot fi întârziate sau pot ajunge în altă ordine. Aplicațiile rulează însă ca **procese**, iar o gazdă rulează de obicei simultan multe procese care comunică (un browser, un client de e-mail, un resolver DNS (Domain Name System – sistemul numelor de domeniu), o sesiune SSH (Secure Shell)). Nivelul transport acoperă această diferență: oferă **comunicație logică, cap-la-cap, între procese** care rulează pe gazde diferite.

Nivelul transport este implementat numai în **sistemele terminale** (gazde), de regulă în nucleul sistemului de operare. Ruterele din interiorul rețelei prelucrează pachetele doar până la nivelul rețea și nu inspectează starea de la nivelul transport. Unitatea de date a nivelului transport se numește **segment** în TCP (Transmission Control Protocol) și **datagramă** în UDP (User Datagram Protocol); ea este transportată ca încărcătură utilă a unui pachet IP (Internet Protocol).

Tabelul 2. *Servicii pe care le poate oferi un protocol de transport*

Serviciu	Ce înseamnă	UDP (User Datagram Protocol)	TCP (Transmission Control Protocol)
Multiplexare / demultiplexare	Multe procese împart aceeași adresă IP (Internet Protocol); porturile livrează datele procesului corect	Da	Da
Detecția erorilor	Sumă de control peste antet + date (+ pseudo-antetul IP (Internet Protocol))	Da (opțională în IPv4 (Internet Protocol versiunea 4), obligatorie în IPv6 (Internet Protocol versiunea 6))	Da (obligatorie)
Livrare fiabilă	Fără pierderi, fără duplicate, livrare în ordine	Nu	Da (ACKs (Acknowledgments – confirmări), retransmisii, numere de secvență)
Gestiunea conexiunii	Stabilirea și închiderea explicită a unei sesiuni	Nu (fără conexiune)	Da (stabilire în trei pași, schimb de FIN (Finish – terminare))
Controlul fluxului	Emițătorul nu depășește capacitatea bufferului receptorului	Nu	Da (fereastra receptorului, rwnd)
Controlul congestiei	Emițătorul nu supraîncarcă rețeaua	Nu	Da (fereastra de congestie, cwnd)
Garanții de timp / lățime de bandă	Întârziere limitată sau debit garantat	Nu	Nu

> **Ideea-cheie**
>
> Nivelul transport transformă livrarea nefiabilă, de la gazdă la gazdă, oferită de IP (Internet Protocol), într-un serviciu pe care aplicațiile îl pot folosi direct. UDP (User Datagram Protocol) adaugă doar minimul necesar (porturi și o sumă de control); TCP (Transmission Control Protocol) adaugă un serviciu complet de flux de octeți fiabil peste o rețea nefiabilă.

2. Protocoale de transport

Suita TCP/IP (Transmission Control Protocol/Internet Protocol – suita de protocoale TCP/IP) definește mai multe protocoale de transport. Două dintre ele transportă aproape tot traficul Internet: **UDP (User Datagram Protocol)** (RFC (Request for Comments – document de standardizare IETF) 768, 1980) și **TCP (Transmission Control Protocol)** (inițial RFC (Request for Comments – document de standardizare IETF) 793, 1981; consolidat în RFC (Request for Comments – document de standardizare IETF) 9293, 2022). Altele există pentru nevoi speciale.

Tabelul 3. *Prezentare generală a protocoalelor de transport*

Protocol	Tip	Caracteristici principale	Utilizare tipică
UDP (User Datagram Protocol)	Fără conexiune, orientat pe mesaje	Antet de 8 octeți, fără fiabilitate, fără ordonare, păstrează limitele mesajelor, permite broadcast/multicast	DNS (Domain Name System – sistemul numelor de domeniu), DHCP (Dynamic Host Configuration Protocol), SNMP (Simple Network Management Protocol), TFTP (Trivial File Transfer Protocol), NTP (Network Time Protocol), VoIP (Voice over Internet Protocol – voce prin IP)/video (RTP (Real-time Transport Protocol)), jocuri online, QUIC (Quick UDP Internet Connections)
TCP (Transmission Control Protocol)	Orientat pe conexiune, flux de octeți	Fiabil, ordonat, full-duplex, control al fluxului, control al congestiei	HTTP/1.1 (Hypertext Transfer Protocol versiunea 1.1) și HTTP/2 (Hypertext Transfer Protocol versiunea 2), HTTPS (Hypertext Transfer Protocol Secure) (TLS (Transport Layer Security)), SMTP (Simple Mail Transfer Protocol), IMAP (Internet Message Access Protocol), SSH (Secure Shell), FTP (File Transfer Protocol), baze de date
SCTP (Stream Control Transmission Protocol)	Orientat pe conexiune, orientat pe mesaje	Fluxuri multiple, multi-homing, mesaje fiabile	Semnalizare în telecomunicații (SIGTRAN (Signaling Transport – transport de semnalizare)), canale de date WebRTC (Web Real-Time Communication – comunicație web în timp real)
DCCP (Datagram Congestion Control Protocol)	Orientat pe conexiune, nefiabil	Control al congestiei fără fiabilitate	Streaming media experimental
QUIC (Quick UDP Internet Connections)	Rulează peste UDP (User Datagram Protocol), implementat în spațiul utilizator	Criptat (TLS (Transport Layer Security) 1.3), fluxuri fără blocare la capul cozii, stabilire în 0-RTT/1-RTT (Round-Trip Time – timp dus-întors), migrarea conexiunii	HTTP/3 (Hypertext Transfer Protocol versiunea 3), browsere moderne și CDNs (Content Delivery Networks – rețele de distribuție a conținutului)

Când alegem UDP (User Datagram Protocol)? Când aplicația preferă promptitudinea în locul fiabilității (voce și video în timp real), când schimbul constă dintr-o singură cerere și un singur răspuns scurte (DNS (Domain Name System – sistemul numelor de domeniu)), când este nevoie de broadcast sau multicast, ori când aplicația vrea să-și implementeze propria fiabilitate și propriul control al congestiei (QUIC (Quick UDP Internet Connections)). **Când alegem TCP (Transmission Control Protocol)?** Când fiecare octet trebuie să ajungă exact

o dată și în ordine (pagini web, fișiere, e-mail, sesiuni de lucru la distanță) și aplicația preferă ca sistemul de operare să se ocupe de fiabilitate.

3. Numere de port, socket-uri și multiplexare

Un **număr de port** este un întreg fără semn pe 16 biți (0–65535) care identifică un punct terminal de comunicație în interiorul unei gazde. Împreună cu o adresă IP (Internet Protocol) și cu un protocol de transport, el identifică un anumit proces (mai exact, un anumit socket deschis de un proces).

Tabelul 4. *Intervalele numerelor de port (IANA (Internet Assigned Numbers Authority – autoritatea de alocare a numerelor Internet), RFC (Request for Comments – document de standardizare IETF) 6335)*

Interval	Denumire	Alocare	Exemple
0 – 1023	Porturi binecunoscute (de sistem)	Alocate de IANA (Internet Assigned Numbers Authority – autoritatea de alocare a numerelor Internet) serviciilor standard; în Linux/Unix asocierea cu ele necesită drepturi de administrator	20/21 FTP (File Transfer Protocol), 22 SSH (Secure Shell), 23 Telnet, 25 SMTP (Simple Mail Transfer Protocol), 53 DNS (Domain Name System – sistemul numelor de domeniu), 67/68 DHCP (Dynamic Host Configuration Protocol), 80 HTTP (Hypertext Transfer Protocol), 110 POP3 (Post Office Protocol versiunea 3), 123 NTP (Network Time Protocol), 143 IMAP (Internet Message Access Protocol), 161 SNMP (Simple Network Management Protocol), 443 HTTPS (Hypertext Transfer Protocol Secure)
1024 – 49151	Porturi înregistrate (de utilizator)	Înregistrate la IANA (Internet Assigned Numbers Authority – autoritatea de alocare a numerelor Internet) pentru aplicații anume	1433 Microsoft SQL (Structured Query Language) Server, 1883 MQTT (Message Queuing Telemetry Transport), 3306 MySQL, 3389 RDP (Remote Desktop Protocol), 5432 PostgreSQL, 8080 HTTP (Hypertext Transfer Protocol) alternativ
49152 – 65535	Porturi dinamice / private / efemere	Nu se alocă niciodată; folosite de OS (Operating System – sistem de operare) pentru partea de client a conexiunilor	Interval efemer implicit în Windows 49152 –65535; în Linux 32768–60999

3.1 Adresa de socket și 5-tuplul

O **adresă de socket** este perechea (adresă IP (Internet Protocol), port), de exemplu `192.168.1.10:51544`. O **conexiune TCP (Transmission Control Protocol)** este identificată în mod unic de **5-tuplu** (protocol, IP (Internet Protocol) sursă, port sursă, IP (Internet Protocol) destinație, port destinație). Deoarece tuplul include și partea clientului, un server web poate gestiona simultan mii de conexiuni pe același port local 443: fiecare conexiune diferă cel puțin prin adresa sau portul clientului.

3.2 Demultiplexarea

- **Demultiplexarea UDP (User Datagram Protocol)** folosește doar destinația (IP (Internet Protocol), port). Toate datagramele trimise către `10.0.0.5:53` ajung la același socket, indiferent de emițător; aplicația află emițătorul din datagrama primită.
- **Demultiplexarea TCP (Transmission Control Protocol)** folosește 5-tuplul complet. Un server are un *socket de ascultare* (de ex. `*:443`) plus câte un *socket conectat* pentru fiecare client; segmentele sosite sunt livrate socket-ului conectat al cărui 5-tuplu se potrivește.
- **Multiplexarea** este operația inversă, la emițător: datele de la multe socket-uri sunt colectate, primesc antete de transport cu porturile corecte și sunt transmise în jos, către IP (Internet Protocol).

Exemplu

Un laptop `192.168.1.10` deschide două file de browser către același server `93.184.215.14:443`. OS (Operating System – sistem de operare) alocă două porturi efemere, de ex. 51544 și 51545. Cele două conexiuni sunt

(TCP (Transmission Control Protocol), 192.168.1.10, 51544, 93.184.215.14, 443) și (TCP (Transmission Control Protocol), 192.168.1.10, 51545, 93.184.215.14, 443).

Ele au aceeași adresă de socket destinație, dar sunt conexiuni diferite, astfel încât TCP (Transmission Control Protocol)-ul serverului le ține separat.

4. UDP (User Datagram Protocol)

UDP (User Datagram Protocol) este cel mai simplu protocol de transport posibil: adaugă peste IP (Internet Protocol) doar multiplexarea pe bază de porturi și o verificare opțională a erorilor. Fiecare operație `send` produce exact o datagramă UDP (User Datagram Protocol), iar fiecare `receive` întoarce exact o datagramă: **limitele mesajelor sunt păstrate**. Nu există stabilirea conexiunii, confirmări, retransmisii, ordonare sau control al congestiei.

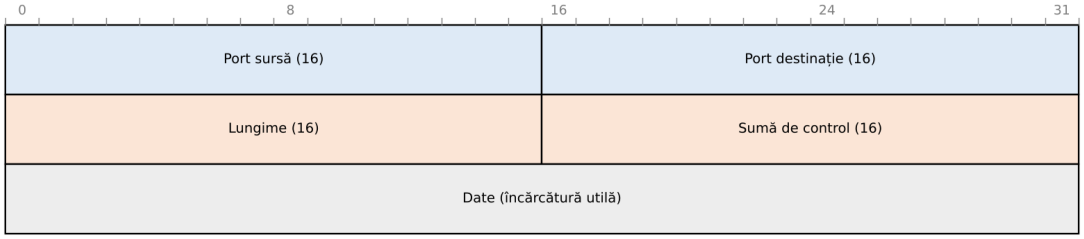


Figura 1. *Formatul antetului UDP (User Datagram Protocol) (8 octeți)*

Tabelul 5. *Câmpurile antetului UDP (User Datagram Protocol)*

Câmp	Dimensiune	Semnificație
Port sursă	16 biți	Portul procesului emițător (poate fi 0 dacă nu se așteaptă răspuns)
Port destinație	16 biți	Portul procesului receptor
Lungime	16 biți	Lungimea antetului + datelor, în octeți (minimum 8); încărcătura utilă maximă este de 65.507 octeți peste IPv4 (Internet Protocol versiunea 4)
Sumă de control	16 biți	Sumă în complement față de unu peste un pseudo-antet (IP (Internet Protocol) sursă/destinație, protocol 17, lungime UDP (User Datagram Protocol)), antetul UDP (User Datagram Protocol) și date. 0 = necalculată (doar IPv4 (Internet Protocol versiunea 4))

Avantaje: nu există întârziere de conectare, antet mic (8 octeți față de cel puțin 20 la TCP (Transmission Control Protocol)), nicio stare de conexiune pe server (scalează la mulți clienți), aplicația controlează exact ce se trimite și când. **Dezavantaje:** pierderile, duplicatele și reordonarea trebuie tratate de aplicație, dacă sunt importante; emițătorii UDP (User Datagram Protocol) care nu își controlează rata pot afecta celălalt trafic.

5. TCP (Transmission Control Protocol)

TCP (Transmission Control Protocol) oferă un **flux de octeți fiabil, ordonat și full-duplex** între două procese. Aplicația scrie un flux de octeți; TCP (Transmission Control Protocol) îl împarte în **segmente**, numerotează fiecare octet, retransmite ce s-a pierdut și livrează octeții în ordine la celălalt capăt. TCP (Transmission Control

Protocol) **nu** păstrează limitele mesajelor: două apeluri `write` de câte 5 octeți pot ajunge ca un singur `read` de 10 octeți sau ca două citiri de 3 și 7 octeți.

- **Orientat pe conexiune:** o stabilire explicită a conexiunii creează stare la ambele capete înainte de transferul datelor.
- **Punct-la-punct și full-duplex:** exact două capete; datele circulă simultan în ambele sensuri.
- **Fiabil:** numere de secvență, confirmări cumulative, sume de control, temporizatoare de retransmisie.
- **Controlul fluxului:** receptorul anunță spațiul liber din bufferul său (*fereastra de recepție*).
- **Controlul congestiei:** emițătorul își limitează rata în funcție de încărcarea estimată a rețelei (*fereastra de congestie*).

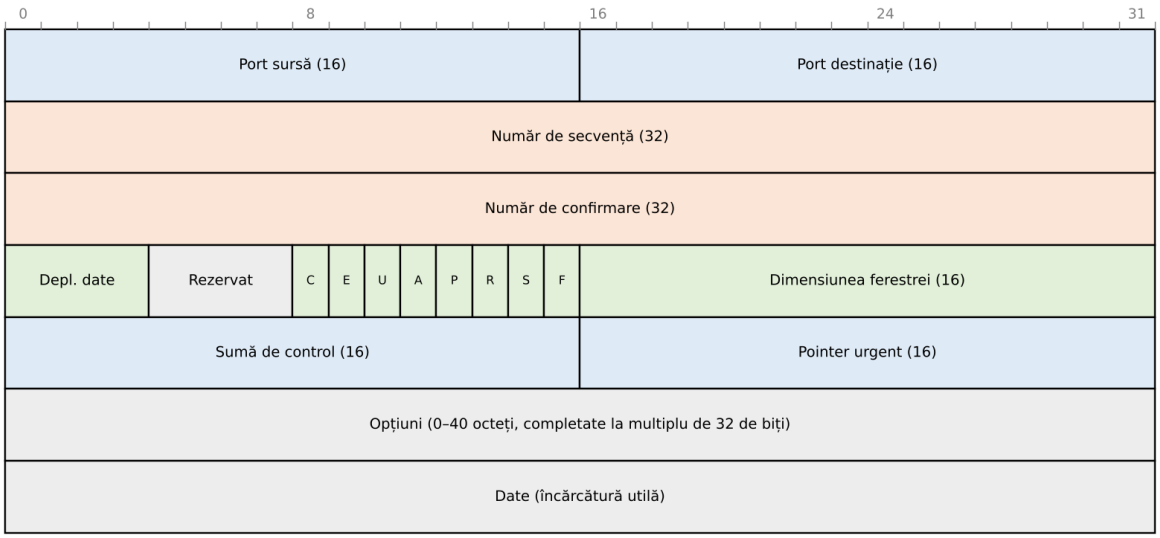


Figura 2. *Formatul antetului TCP (Transmission Control Protocol) (20 de octeți fără opțiuni, maximum 60 de octeți)*

Tabelul 6. *Câmpurile antetului TCP (Transmission Control Protocol)*

Câmp	Dimensiune	Semnificație
Port sursă / destinație	16 + 16 biți	Identifică socket-urile emițător și receptor
Număr de secvență	32 de biți	Numărul primului octet de date din acest segment. Într-un SYN (Synchronize – sincronizare) conține ISN (Initial Sequence Number – număr de secvență inițial)
Număr de confirmare	32 de biți	Următorul octet pe care emițătorul acestui segment se așteaptă să-l primească (ACK (Acknowledgment – confirmare) cumulativ); valid când indicatorul ACK (Acknowledgment – confirmare) = 1
Deplasament date	4 biți	Lungimea antetului în cuvinte de 32 de biți (5–15, adică 20–60 de octeți)
Indicatori	8 biți	CWR (Congestion Window Reduced – fereastră de congestie redusă), ECE (ECN-Echo – ecou al notificării explicite de congestie) (notificare explicită a congestiei), URG (Urgent – urgent), ACK (Acknowledgment – confirmare), PSH (Push – livrare imediată), RST (Reset – resetare) (resetare/abandon), SYN (Synchronize – sincronizare) (sincronizare / deschidere), FIN (Finish – terminare) (terminare / închidere)
Dimensiunea ferestrei	16 biți	Fereastra de recepție: câți octeți, după cel confirmat, poate accepta receptorul (scalată prin opțiunea Window Scale)
Sumă de control	16 biți	Acoperă pseudo-antetul, antetul TCP (Transmission Control Protocol) și datele (obligatorie)
Pointer urgent	16 biți	Deplasamentul datelor urgente când URG (Urgent – urgent) = 1 (rar folosit)

Câmp	Dimensiune	Semnificație
Opțiuni	0–40 de octeți	MSS (Maximum Segment Size – dimensiunea maximă a segmentului), Window Scale, SACK (Selective Acknowledgment – confirmare selectivă)-permitted / blocuri SACK (Selective Acknowledgment – confirmare selectivă), Timestamps

MSS (Maximum Segment Size – dimensiunea maximă a segmentului) este cantitatea maximă de *date* dintr-un segment. Pe Ethernet (MTU (Maximum Transmission Unit – unitatea maximă de transmisie) 1500 de octeți), cu IPv4 (Internet Protocol versiunea 4) și fără opțiuni, MSS (Maximum Segment Size – dimensiunea maximă a segmentului) = 1500 − 20 (IP (Internet Protocol)) − 20 (TCP (Transmission Control Protocol)) = **1460 de octeți**. Fiecare parte își anunță MSS (Maximum Segment Size – dimensiunea maximă a segmentului) în segmentul SYN (Synchronize – sincronizare).

6. Mecanisme TCP (Transmission Control Protocol)

6.1 Stabilirea conexiunii: procedura în trei pași (three-way handshake)

Înainte de schimbul de date, ambele capete trebuie să convină că există o conexiune și trebuie să afle fiecare **ISN (Initial Sequence Number – număr de secvență inițial)** al celuilalt, dimensiunea ferestrei și opțiunile. TCP (Transmission Control Protocol) face acest lucru cu trei segmente:

1. **SYN (Synchronize – sincronizare)** — clientul (cel care deschide activ) trimite un segment cu SYN (Synchronize – sincronizare) = 1 și `seq = x` (ISN (Initial Sequence Number – număr de secvență inițial) propriu). Clientul trece în starea SYN_SENT.
2. **SYN (Synchronize – sincronizare) + ACK (Acknowledgment – confirmare)** — serverul (care deschide pasiv, în starea LISTEN) răspunde cu SYN (Synchronize – sincronizare) = 1, ACK (Acknowledgment – confirmare) = 1, `seq = y` (ISN (Initial Sequence Number – număr de secvență inițial) propriu) și `ack = x + 1`. Serverul trece în SYN_RCVD.
3. **ACK (Acknowledgment – confirmare)** — clientul confirmă cu ACK (Acknowledgment – confirmare) = 1, `seq = x + 1`, `ack = y + 1`. Ambele capete sunt acum ESTABLISHED și datele pot circula.

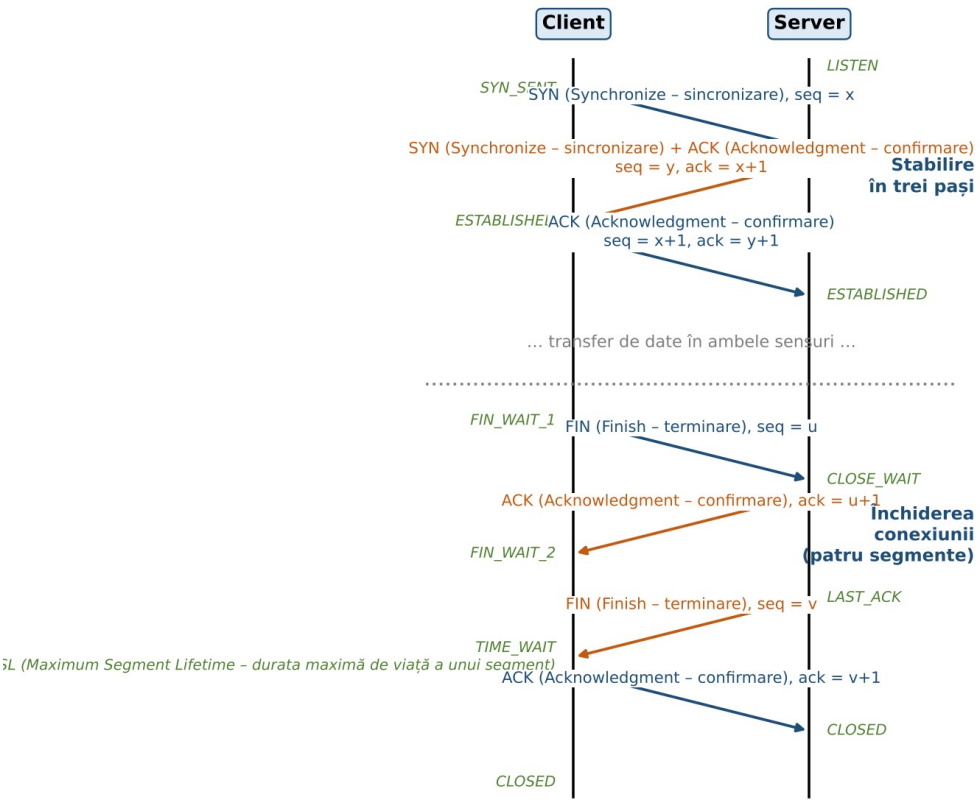


Figura 3. *Stabilirea conexiunii TCP (Transmission Control Protocol) în trei pași și închiderea în patru segmente, cu stările TCP (Transmission Control Protocol) ale fiecărui capăt*

De ce trei segmente și nu două? Fiecare parte trebuie să-și vadă confirmat propriul ISN (Initial Sequence Number – număr de secvență inițial). Cu două segmente, serverul nu ar ști sigur dacă clientul i-a primit SYN (Synchronize – sincronizare)-ul, iar un SYN (Synchronize – sincronizare) duplicat întârziat dintr-o conexiune veche ar putea deschide o conexiune falsă. Indicatorii SYN (Synchronize – sincronizare) și FIN (Finish – terminare) consumă fiecare **un număr de secvență**, de aceea confirmările conțin x + 1 și y + 1.

De ce ISNs (Initial Sequence Numbers – numere de secvență inițiale) aleatoare? ISNs (Initial Sequence Numbers – numere de secvență inițiale) se aleg pseudo-aleator (RFC (Request for Comments – document de standardizare IETF) 6528), astfel încât segmentele dintr-o conexiune veche cu același 5-tuplu să aibă șanse mici să cadă în spațiul de secvență al noii conexiuni și astfel încât atacatorii să nu poată ghici ușor numerele de secvență pentru a injecta date.

6.2 Închiderea conexiunii și stările TCP (Transmission Control Protocol)

Deoarece TCP (Transmission Control Protocol) este full-duplex, fiecare sens se închide independent. Partea care închide prima trimite **FIN (Finish – terminare)**; cealaltă parte îl confirmă, mai poate trimite date (conexiune semi-închisă) și apoi trimite propriul FIN (Finish – terminare), care este confirmat. Partea care a trimis primul FIN (Finish – terminare) așteaptă în **TIME_WAIT** timp de 2·MSL (Maximum Segment Lifetime – durata maximă de viață a unui segment) (în practică între 30 s și 4 min; 60 s în Linux), pentru ca un ACK (Acknowledgment – confirmare) final pierdut să poată fi retransmis și pentru ca duplicatele vechi să expire. O conexiune poate fi și întreruptă brusc cu **RST (Reset – resetare)** — de exemplu când un SYN (Synchronize – sincronizare) sosește pe un port pe care nu ascultă niciun proces.

Tabelul 7. *Principalele stări ale unei conexiuni TCP (Transmission Control Protocol) (vizibile cu netstat / ss)*

Stare	Semnificație
LISTEN	Serverul așteaptă cereri de conexiune
SYN_SENT / SYN_RCVD	Stabilirea conexiunii în curs (partea client / server)

Stare	Semnificație
ESTABLISHED	Conexiune deschisă; transfer de date
FIN_WAIT_1 / FIN_WAIT_2	Partea care închide activ a trimis FIN (Finish – terminare); așteaptă ACK (Acknowledgment – confirmare) și apoi FIN (Finish – terminare)-ul celuilalt capăt
CLOSE_WAIT	Partea care închide pasiv a primit FIN (Finish – terminare); așteaptă ca aplicația locală să închidă
LAST_ACK	Partea care închide pasiv a trimis propriul FIN (Finish – terminare); așteaptă ACK (Acknowledgment – confirmare)-ul final
TIME_WAIT	Partea care închide activ așteaptă 2·MSL (Maximum Segment Lifetime – durata maximă de viață a unui segment) înainte de a elibera 5-tuplul
CLOSED	Nicio conexiune

Observație frecventă în laborator

Multe conexiuni în **CLOSE_WAIT** pe un server indică de obicei o eroare în aplicație: celălalt capăt a închis, dar programul nu apelează niciodată `close()` pe socket. Multe conexiuni în **TIME_WAIT** sunt normale pe un client încărcat sau pe partea care închide prima.

6.3 Transferul fiabil: transmisia stop-and-wait

Cel mai simplu protocol fiabil peste un canal nefiabil este **ARQ (Automatic Repeat reQuest – cerere automată de repetare) stop-and-wait**. Emițătorul trimite un cadru (segment), pornește un temporizator și așteaptă. Receptorul verifică cadrul și trimite înapoi o confirmare. Dacă ACK (Acknowledgment – confirmare) sosește înainte să expire temporizatorul, emițătorul trimite cadrul următor; altfel retransmite același cadru. Un **număr de secvență pe 1 bit** (0, 1, 0, 1, …) este suficient pentru ca receptorul să recunoască duplicatele cauzate de ACKs (Acknowledgments – confirmări) pierdute.

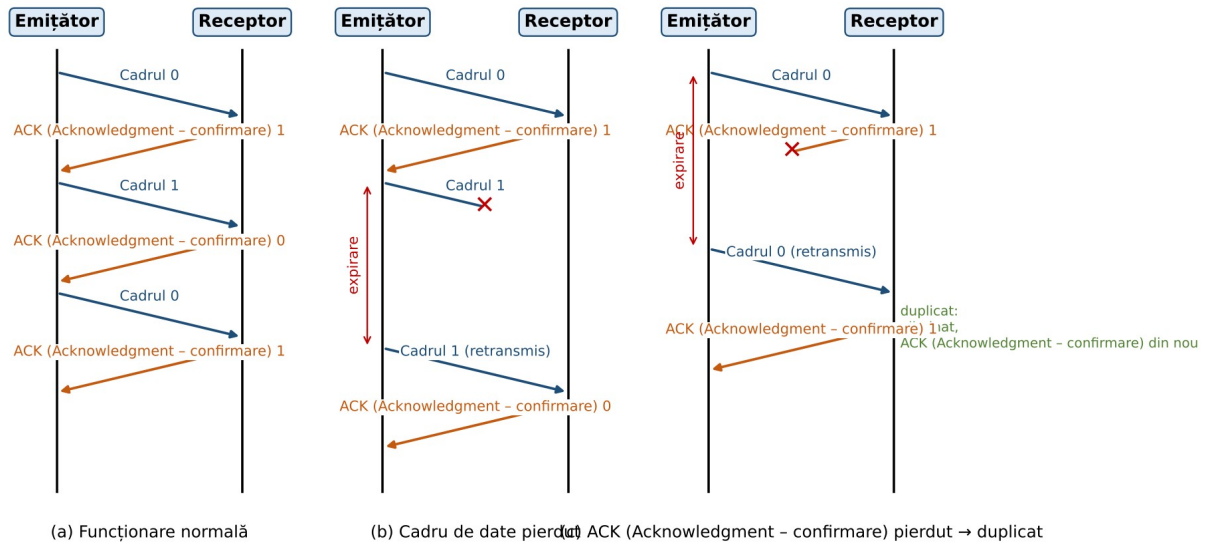


Figura 4. *ARQ (Automatic Repeat reQuest – cerere automată de repetare) stop-and-wait: (a) funcționare normală, (b) cadru pierdut, recuperat prin expirarea temporizatorului, (c) ACK (Acknowledgment – confirmare) pierdut, care produce un duplicat eliminat de receptor*

Stop-and-wait este corect, dar ineficient pe legături cu întârziere mare sau debit mare, deoarece emițătorul stă inactiv cât așteaptă fiecare ACK (Acknowledgment – confirmare). Fie **T_t = L / R** timpul de transmisie al unui cadru (L biți pe o legătură de R bit/s) și **RTT (Round-Trip Time – timp dus-întors)** timpul de propagare dus-întors. Dacă timpul de transmisie al ACK (Acknowledgment – confirmare) este neglijabil, **gradul de utilizare** al emițătorului este:

Gradul de utilizare la stop-and-wait

U = T_t / (T_t + RTT (Round-Trip Time – timp dus-întors)) = 1 / (1 + 2a), unde a = T_prop / T_t

Debit = U · R

Exemplu: R = 1 Gbit/s, L = 1500 de octeți = 12.000 de biți, RTT (Round-Trip Time – timp dus-întors) = 30 ms. Atunci T_t = 12 µs și U = 0,012 / 30,012 ≈ **0,04 %** — o legătură de 1 Gbit/s transportă doar aproximativ 400 kbit/s. Aceasta motivează **transmisia în bandă (pipelining)**.

6.4 Transmisia cu fereastră glisantă

La transmisia cu **fereastră glisantă** (în bandă), emițătorul poate avea până la **W** cadre trimise dar încă neconfirmate. Fereastra acoperă un interval de numere de secvență; când cel mai vechi cadru neconfirmat este confirmat, fereastra **glisează** înainte și pot fi trimise cadre noi. Atât emițătorul, cât și receptorul păstrează ferestre: fereastra emițătorului arată ce se poate trimite, fereastra receptorului arată ce se poate accepta.

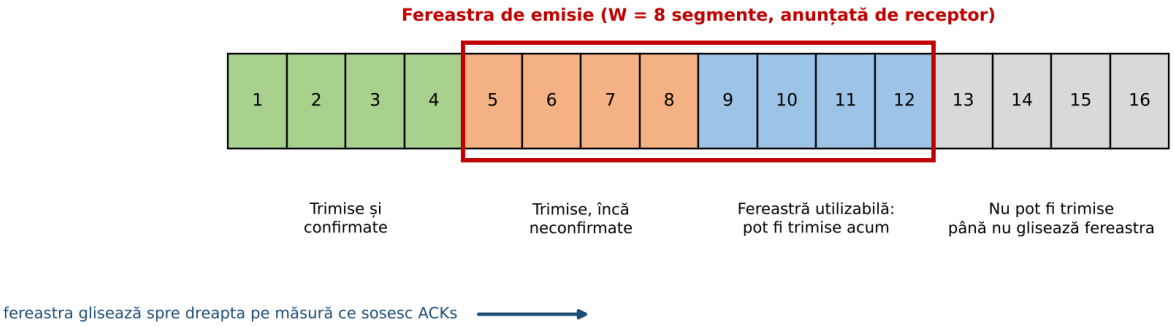


Figura 5. *Spațiul numerelor de secvență văzut de emițător, cu o fereastră glisantă de W = 8 segmente*

Gradul de utilizare cu fereastră glisantă

U = min(1, W · T_t / (T_t + RTT (Round-Trip Time – timp dus-întors)))

Pentru ca legătura să fie complet ocupată: W ≥ (T_t + RTT (Round-Trip Time – timp dus-întors)) / T_t, adică fereastra trebuie să acopere **produsul lățime de bandă × întârziere** R · RTT (Round-Trip Time – timp dus-întors).

Cu ferestrele glisante se folosesc două strategii clasice de recuperare a erorilor:

Tabelul 8. *Go-Back-N comparativ cu Selective Repeat*

	Go-Back-N (GBN (Go-Back-N – întoarcere cu N))	Selective Repeat (SR (Selective Repeat – repetare selectivă))
Fereastra receptorului	1 — cadrele sosite în altă ordine sunt eliminate	W — cadrele sosite în altă ordine sunt memorate
Confirmări	Cumulative (ACK (Acknowledgment – confirmare) n = toate cadrele dinaintea lui n au fost primite)	Individuale, pentru fiecare cadru
La pierdere / expirare	Se retransmit cadrul pierdut și toate cadrele trimise după el	Se retransmite doar cadrul pierdut
Numere de secvență (k biți)	W ≤ 2^k − 1	W ≤ 2^(k−1)
Complexitate	Receptor simplu, risipitor pe legături cu pierderi	Mai multă memorare și logică, eficient

6.5 Cum folosește TCP (Transmission Control Protocol) fereastra glisantă

- TCP (Transmission Control Protocol) numerotează **octeți**, nu segmente; fereastra este exprimată în octeți.
- Confirmările sunt **cumulative** (ca la GBN (Go-Back-N – întoarcere cu N)), dar receptorii memorează datele sosite în altă ordine, iar opțiunea **SACK (Selective Acknowledgment – confirmare selectivă)** (RFC (Request for Comments – document de standardizare IETF) 2018) le permite să raporteze exact ce blocuri au sosit, astfel încât emițătorul retransmite doar ce lipsește (ca la SR (Selective Repeat – repetare selectivă)).
- Receptorul anunță spațiul liber din buffer în câmpul **Window** (**rwnd**) — acesta este **controlul fluxului**. Un receptor cu bufferul plin anunță o **fereastră zero**; emițătorul trimite atunci mici *sonde de fereastră* până când se eliberează spațiu.
- Deoarece câmpul de 16 biți limitează rwnd la 65.535 de octeți, opțiunea **Window Scale** (RFC (Request for Comments – document de standardizare IETF) 7323) îl înmulțește cu 2^S (S ≤ 14), permițând ferestre de până la aproximativ 1 GiB (gibioctet) pe căi cu produs lățime de bandă × întârziere mare.
- Cantitatea de date neconfirmate pe care emițătorul o poate avea în tranzit este **min(rwnd, cwnd)**, unde cwnd este fereastra de congestie (secțiunea 7).

6.6 Temporizarea retransmisiei și retransmisia rapidă

TCP (Transmission Control Protocol) măsoară timpul dus-întors și își adaptează **RTO (Retransmission Timeout – temporizare de retransmisie)** folosind algoritmul din RFC (Request for Comments – document de standardizare IETF) 6298 (α = 1/8, β = 1/4):

Estimarea RTO (Retransmission Timeout – temporizare de retransmisie) (RFC (Request for Comments – document de standardizare IETF) 6298) — srtt = SRTT (Smoothed Round-Trip Time – timp dus-întors netezit), rttvar = RTTVAR (Round-Trip Time Variation – variația timpului dus-întors), rto = RTO (Retransmission Timeout – temporizare de retransmisie)

```
primul eșantion R:   srtt = R;   rttvar = R / 2
eșantioanele următoare R:  rttvar = (1 - β)·rttvar + β·|srtt - R|
                  srtt   = (1 - α)·srtt   + α·R
                  rto    = srtt + max(G, 4·rttvar)     (G = granularitatea ceasului)
la expirare:       rto = 2·rto   (dublare exponențială)
```

Așteptarea expirării temporizatorului este lentă. Dacă emițătorul primește **trei ACKs (Acknowledgments – confirmări) duplicate** pentru același octet, conchide că segmentul următor s-a pierdut (segmentele de după el sosesc) și îl retransmite imediat — **retransmisia rapidă**.

7. Congestia traficului și controlul congestiei în TCP (Transmission Control Protocol)

7.1 Ce este congestia?

Congestia apare când rata totală cu care sursele trimit într-o parte a rețelei depășește capacitatea unei legături sau a unui ruter. Cozile ruterelor se umplu, pachetele așteaptă mai mult (crește întârzierea), iar când bufferele se umplu pachetele sunt **aruncate**. Emițătorii retransmit atunci, adăugând și mai multă încărcare. În cazul extrem, rețeaua transportă în principal retransmisii, iar debitul util se prăbușește — **colapsul de congestie** observat în Internetul timpuriu în 1986, care l-a determinat pe Van Jacobson să proiecteze controlul congestiei în TCP (Transmission Control Protocol) (1988).

Tabelul 9. *Costurile congestiei*

Simptom	Cauză	Consecință
Întârziere de așteptare în coadă tot mai mare	Rata de sosire aproape de capacitatea legăturii	RTT (Round-Trip Time – timp dus-întors) mai mare, interactivitate slabă, posibile expirări premature

Simptom	Cauză	Consecință
Pierderi de pachete	Depășirea bufferului unui ruter	Retransmisii; capacitate irosită pe legăturile anterioare
Retransmisii inutile	Pachetele întârziate (nu pierdute) depășesc temporizarea	Copiile duplicate consumă lățime de bandă
Prăbușirea debitului	Cea mai mare parte a capacității este folosită de pachete aruncate ulterior	Rețeaua livrează puține date utile

Controlul fluxului față de controlul congestiei: controlul fluxului protejează **receptorul** (rwnd, anunțată de receptor); controlul congestiei protejează **rețeaua** (cwnd, estimată de emițător). TCP (Transmission Control Protocol) trimite cel mult min(rwnd, cwnd) octeți neconfirmați.

7.2 Abordări

- **Controlul congestiei cap-la-cap** — rețeaua nu oferă niciun semnal explicit; sistemele terminale deduc congestia din pierderi și întârzieri. Aceasta este abordarea TCP (Transmission Control Protocol) clasică.
- **Controlul congestiei asistat de rețea** — ruterele semnalează congestia, de ex. **ECN (Explicit Congestion Notification – notificare explicită a congestiei)** (RFC (Request for Comments – document de standardizare IETF) 3168): un ruter marchează antetul IP (Internet Protocol) în loc să arunce pachetul; receptorul trimite înapoi marcajul prin indicatorul TCP (Transmission Control Protocol) ECE (ECN-Echo – ecou al notificării explicite de congestie); emițătorul reduce cwnd și confirmă cu CWR (Congestion Window Reduced – fereastră de congestie redusă).
- **Gestiunea cozilor în rutere** — AQM (Active Queue Management – gestiune activă a cozilor) (RED (Random Early Detection – detecție timpurie aleatoare), CoDel (Controlled Delay – întârziere controlată), fq_codel) aruncă sau marchează pachete din timp, pentru a menține cozile scurte.

7.3 Algoritmii TCP (Transmission Control Protocol) de control al congestiei (Tahoe / Reno, RFC (Request for Comments – document de standardizare IETF) 5681)

Emițătorul păstrează două variabile: **fereastra de congestie cwnd** (în octeți sau în unități MSS (Maximum Segment Size – dimensiunea maximă a segmentului)) și **pragul de pornire lentă ssthresh**. Principiul este **AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă)**: se sondează cu prudență pentru mai multă lățime de bandă cât timp nu apar pierderi și se reduce brusc rata când pierderile semnalează congestie.

Tabelul 10. *Regulile de control al congestiei în TCP (Transmission Control Protocol) Reno*

Fază / eveniment	Regulă	Efect
Pornire lentă (cwnd < ssthresh)	cwnd += 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) pentru fiecare ACK (Acknowledgment – confirmare) primit	cwnd se dublează la fiecare RTT (Round-Trip Time – timp dus-întors) (creștere exponențială); cwnd inițial este de până la 10 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) pe sistemele moderne (RFC (Request for Comments – document de standardizare IETF) 6928)
Evitarea congestiei (cwnd ≥ ssthresh)	cwnd += MSS (Maximum Segment Size – dimensiunea maximă a segmentului)·MSS (Maximum Segment Size – dimensiunea maximă a segmentului)/cwnd pentru fiecare ACK (Acknowledgment – confirmare)	cwnd crește cu ≈ 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) la fiecare RTT (Round-Trip Time – timp dus-întors) (creștere aditivă)
3 ACKs (Acknowledgments – confirmări) duplicate (Reno)	ssthresh = cwnd/2; retransmisie rapidă; cwnd = ssthresh (+3 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) în timpul recuperării rapide)	Reacție moderată: rețeaua încă livrează pachete

Fază / eveniment	Regulă	Efect
Expirarea temporizatorului de retransmisie	ssthresh = cwnd/2; cwnd = 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului); revenire la pornirea lentă	Reacție puternică: se presupune congestie severă
TCP (Transmission Control Protocol) Tahoe (mai vechi)	Orice pierdere → cwnd = 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului), pornire lentă	Fără recuperare rapidă

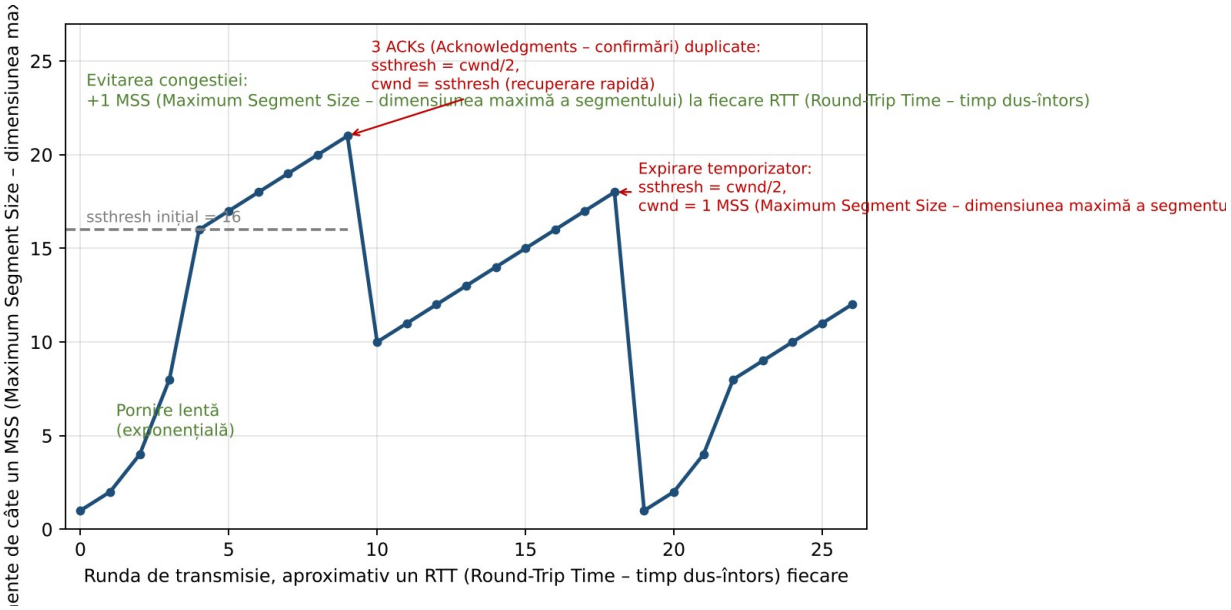


Figura 6. *Evoluția cwnd pentru TCP (Transmission Control Protocol) Reno: pornire lentă, evitarea congestiei, recuperare rapidă după trei ACKs (Acknowledgments – confirmări) duplicate (runda 9) și expirarea temporizatorului (runda 18)*

7.4 Algoritmi moderni

- **CUBIC** (RFC (Request for Comments – document de standardizare IETF) 9438) — cwnd crește ca funcție cubică de timpul scurs de la ultima pierdere, ceea ce funcționează mult mai bine pe căi cu produs lățime de bandă × întârziere mare. Implicit în Linux de la nucleul 2.6.19 și în Windows de la Windows 10 versiunea 1709 / Server 2019.
- **Compound TCP (Transmission Control Protocol)** — algoritmul implicit mai vechi din Windows, care combină o fereastră bazată pe pierderi cu una bazată pe întârziere.
- **BBR (Bottleneck Bandwidth and Round-trip propagation time)** (Google) — bazat pe model: estimează lățimea de bandă a gâtuirii și RTT (Round-Trip Time – timp dus-întors) minim și dozează ritmul pachetelor în consecință, în loc să reacționeze doar la pierderi. Disponibil în Linux (modulul `tcp_bbr`).
- Implementările **QUIC (Quick UDP Internet Connections)** rulează controlul congestiei în spațiul utilizator (de obicei CUBIC sau BBR (Bottleneck Bandwidth and Round-trip propagation time)).

Echitate: cu AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă), N fluxuri TCP (Transmission Control Protocol) care împart o gâtuire converg către o cotă egală R/N fiecare. Aplicațiile pot ocoli echitatea deschizând multe conexiuni paralele, iar aplicațiile UDP (User Datagram Protocol) fără control al congestiei nu sunt deloc limitate — unul dintre motivele pentru care aplicațiile în timp real trebuie să-și implementeze propriul control al ratei.

8. Rezumat și termeni-cheie

- Nivelul transport oferă comunicație de la proces la proces; rulează doar în sistemele terminale.
- Porturile (16 biți) împreună cu adresele IP (Internet Protocol) identifică socket-urile; o conexiune TCP (Transmission Control Protocol) este identificată prin 5-tuplul său.

- UDP (User Datagram Protocol): fără conexiune, antet de 8 octeți, orientat pe mesaje, fără fiabilitate — rapid și simplu.
- TCP (Transmission Control Protocol): flux de octeți fiabil, orientat pe conexiune, antet de 20–60 de octeți, numerele de secvență numără octeți.
- Stabilirea conexiunii în trei pași (SYN (Synchronize – sincronizare), SYN-ACK (Synchronize-Acknowledgment – sincronizare și confirmare), ACK (Acknowledgment – confirmare)) sincronizează ISNs (Initial Sequence Numbers – numere de secvență inițiale); schimbul FIN (Finish – terminare)/ACK (Acknowledgment – confirmare) închide fiecare sens; TIME_WAIT protejează împotriva duplicatelor vechi.
- Stop-and-wait este simplu, dar ineficient (U = T_t/(T_t + RTT (Round-Trip Time – timp dus-întors))); ferestrele glisante umplu legătura când W ≥ produsul lățime de bandă × întârziere.
- Controlul fluxului (rwnd) protejează receptorul; controlul congestiei (cwnd: pornire lentă, evitarea congestiei, retransmisie/recuperare rapidă, AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă)) protejează rețeaua.

Termeni-cheie: segment, datagramă, port, port efemer, adresă de socket, 5-tuplu, multiplexare, demultiplexare, ISN (Initial Sequence Number – număr de secvență inițial), SYN (Synchronize – sincronizare), ACK (Acknowledgment – confirmare), FIN (Finish – terminare), RST (Reset – resetare), MSS (Maximum Segment Size – dimensiunea maximă a segmentului), MTU (Maximum Transmission Unit – unitatea maximă de transmisie), RTT (Round-Trip Time – timp dus-întors), RTO (Retransmission Timeout – temporizare de retransmisie), ARQ (Automatic Repeat reQuest – cerere automată de repetare), Go-Back-N, Selective Repeat, SACK (Selective Acknowledgment – confirmare selectivă), rwnd, cwnd, ssthresh, AIMD (Additive Increase, Multiplicative Decrease – creștere aditivă, scădere multiplicativă), pornire lentă, retransmisie rapidă, recuperare rapidă, ECN (Explicit Congestion Notification – notificare explicită a congestiei), CUBIC, BBR (Bottleneck Bandwidth and Round-trip propagation time).

Partea a II-a — Seminar: probleme rezolvate și exerciții

Seminarul aplică noțiunile de la curs în probleme numerice și de urmărire. Problemele rezolvate se parcurg la seminar; exercițiile propuse sunt pentru lucru individual și temă.

Problema 1 — Porturi, socket-uri și demultiplexare

Gazda A (`10.0.0.5`) rulează un server web pe portul TCP (Transmission Control Protocol) 80 și un server DNS (Domain Name System – sistemul numelor de domeniu) pe portul UDP (User Datagram Protocol) 53. Gazda B (`10.0.0.20`) deschide două conexiuni TCP (Transmission Control Protocol) către serverul web, din porturile 50001 și 50002, și trimite o interogare DNS (Domain Name System – sistemul numelor de domeniu) din portul UDP (User Datagram Protocol) 50001. Gazda C (`10.0.0.30`) deschide o conexiune TCP (Transmission Control Protocol) către serverul web, tot din portul 50001.

1. Câte socket-uri folosește gazda A și ce identifică fiecare dintre ele?
2. Poate B să folosească portul 50001 simultan pentru TCP (Transmission Control Protocol) și UDP (User Datagram Protocol)? Pot B și C să folosească amândouă portul de client 50001?
3. Ce câmpuri folosește gazda A pentru a livra un segment sosit socket-ului corect?

> **Rezolvare**
>
> 1. Gazda A folosește **5 socket-uri**: 1 socket TCP (Transmission Control Protocol) de ascultare `*:80`, 3 socket-uri TCP (Transmission Control Protocol) conectate — (10.0.0.5:80, 10.0.0.20:50001), (10.0.0.5:80, 10.0.0.20:50002), (10.0.0.5:80, 10.0.0.30:50001) — și 1 socket UDP (User Datagram Protocol) `*:53`.
>
> 2. Da: TCP (Transmission Control Protocol) și UDP (User Datagram Protocol) au spații de porturi separate. Da: conexiunile de la B și C diferă prin IP (Internet Protocol)-ul sursă, deci 5-tuplurile lor sunt diferite.
>
> 3. Pentru TCP (Transmission Control Protocol): 5-tuplul complet (protocol, IP (Internet Protocol) sursă, port sursă, IP (Internet Protocol) destinație, port destinație). Pentru UDP (User Datagram Protocol): doar IP (Internet Protocol)-ul și portul destinație.

Problema 2 — Eficiența transmisiei stop-and-wait

O legătură are debitul R = 10 Mbit/s și o întârziere de propagare într-un sens de 10 ms. Cadrele conțin L = 1250 de octeți. ACKs (Acknowledgments – confirmări) sunt foarte scurte (timpul lor de transmisie poate fi neglijat). Calculați gradul de utilizare și debitul efectiv în cazul stop-and-wait.

> **Rezolvare**
>
> T_t = L / R = 10.000 de biți / 10^7 bit/s = **1 ms**. RTT (Round-Trip Time – timp dus-întors) = 2 · 10 ms = **20 ms**.
>
> U = T_t / (T_t + RTT (Round-Trip Time – timp dus-întors)) = 1 / 21 ≈ **4,76 %**.
>
> Debit = U · R ≈ 0,0476 · 10 Mbit/s ≈ **476 kbit/s**.

Problema 3 — Fereastră glisantă pe aceeași legătură

Folosind legătura din problema 2: (a) Ce dimensiune a ferestrei W (în cadre) este necesară pentru o utilizare de 100 %? (b) Care sunt gradul de utilizare și debitul pentru W = 8? (c) De câți biți de număr de secvență este nevoie pentru fereastra de la (a), la Go-Back-N și la Selective Repeat?

Rezolvare

(a) W ≥ (T_t + RTT (Round-Trip Time – timp dus-întors)) / T_t = 21 / 1 = **21 de cadre**.

(b) U = 8 · 1 / 21 ≈ **38,1 %**; debit ≈ **3,81 Mbit/s**.

(c) Go-Back-N: W ≤ 2^k − 1 → 2^k ≥ 22 → **k = 5 biți**. Selective Repeat: W ≤ 2^(k−1) → 2^(k−1) ≥ 21 → **k = 6 biți**.

Problema 4 — Produsul lățime de bandă × întârziere și fereastra TCP (Transmission Control Protocol)

O conexiune TCP (Transmission Control Protocol) rulează pe o cale cu R = 100 Mbit/s și RTT (Round-Trip Time – timp dus-întors) = 80 ms. (a) Calculați produsul lățime de bandă × întârziere. (b) Care este debitul maxim dacă fereastra de recepție este limitată la 65.535 de octeți (fără scalarea ferestrei)? (c) Ce deplasare Window Scale este necesară pentru a umple legătura?

Rezolvare

(a) BDP (Bandwidth-Delay Product – produsul lățime de bandă × întârziere) = R · RTT (Round-Trip Time – timp dus-întors) = 10^8 · 0,08 = 8 · 10^6 biți = **1.000.000 de octeți**.

(b) Cel mult o fereastră pe RTT (Round-Trip Time – timp dus-întors): 65.535 · 8 biți / 0,08 s ≈ **6,55 Mbit/s** (doar 6,5 % din legătură).

(c) 1.000.000 / 65.535 ≈ 15,3 → avem nevoie de un factor ≥ 16 = 2^4, deci **deplasarea S = 4** (fereastră maximă 1.048.560 de octeți).

Problema 5 — Urmărirea numerelor de secvență și de confirmare

Un client cu ISN (Initial Sequence Number – număr de secvență inițial) = 1000 se conectează la un server cu ISN (Initial Sequence Number – număr de secvență inițial) = 5000. După stabilirea conexiunii, clientul trimite 200 de octeți, apoi 300 de octeți; serverul îi confirmă pe amândoi printr-un singur ACK (Acknowledgment – confirmare), apoi trimite un răspuns de 500 de octeți. Clientul confirmă răspunsul și închide conexiunea, urmat de server. Completați indicatorii, numerele de secvență și de confirmare pentru fiecare segment.

Tabelul 11. *Rezolvarea problemei 5*

#	Sens	Indicatori	seq	ack	Date (octeți)
1	C → S	SYN (Synchronize – sincronizare)	1000	—	0
2	S → C	SYN (Synchronize – sincronizare), ACK (Acknowledgment – confirmare)	5000	1001	0
3	C → S	ACK (Acknowledgment – confirmare)	1001	5001	0
4	C → S	ACK (Acknowledgment – confirmare), PSH (Push – livrare imediată)	1001	5001	200
5	C → S	ACK (Acknowledgment – confirmare), PSH (Push – livrare imediată)	1201	5001	300
6	S → C	ACK (Acknowledgment – confirmare)	5001	1501	0

#	Sens	Indicatori	seq	ack	Date (octeți)
7	S → C	ACK (Acknowledgment – confirmare), PSH (Push – livrare imediată)	5001	1501	500
8	C → S	ACK (Acknowledgment – confirmare)	1501	5501	0
9	C → S	FIN (Finish – terminare), ACK (Acknowledgment – confirmare)	1501	5501	0
10	S → C	ACK (Acknowledgment – confirmare)	5501	1502	0
11	S → C	FIN (Finish – terminare), ACK (Acknowledgment – confirmare)	5501	1502	0
12	C → S	ACK (Acknowledgment – confirmare)	1502	5502	0

Observați că SYN (Synchronize – sincronizare) și FIN (Finish – terminare) consumă fiecare câte un număr de secvență, în timp ce ACKs (Acknowledgments – confirmări) simple nu consumă niciunul. În practică, segmentele 6 și 7 sunt adesea combinate (ACK (Acknowledgment – confirmare) este *atașat* datelor — piggybacking).

Problema 6 — Evoluția ferestrei de congestie (TCP (Transmission Control Protocol) Reno)

Un emițător TCP (Transmission Control Protocol) Reno pornește cu cwnd = 1 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) și ssthresh = 16 MSS (Maximum Segment Size – dimensiunea maximă a segmentului). La sfârșitul rundei 9 se primesc trei ACKs (Acknowledgments – confirmări) duplicate, iar la sfârșitul rundei 18 expiră temporizatorul (vezi figura 6). Răspundeți: (a) În ce runde funcționează pornirea lentă? (b) Cât sunt ssthresh și cwnd după fiecare eveniment de pierdere? (c) Câte segmente se trimit în rundele 0–4? (d) Cât este cwnd în runda 22?

Tabelul 12. *cwnd pe runde (în MSS (Maximum Segment Size – dimensiunea maximă a segmentului))*

Runda	0	1	2	3	4	5	…	9	10	…	18	19	20	21	22	23
cwnd	1	2	4	8	16	17	…	21	10	…	18	1	2	4	8	9

Rezolvare

(a) Pornire lentă: rundele 0–4 (cwnd 1→16 atinge ssthresh) și rundele 19–22, după expirarea temporizatorului.

(b) După 3 ACKs (Acknowledgments – confirmări) duplicate la cwnd = 21: ssthresh = $\lfloor 21/2 \rfloor = 10$, cwnd = 10 (recuperare rapidă, apoi evitarea congestiei). După expirarea temporizatorului la cwnd = 18: ssthresh = 9, cwnd = 1.

(c) 1 + 2 + 4 + 8 + 16 = **31 de segmente**.

(d) Runda 22: cwnd = 8 (pornirea lentă dublează 1, 2, 4, 8); în runda 23 dublarea ar da 16 > ssthresh, deci cwnd este limitat la 9 și începe evitarea congestiei.

Problema 7 — Calculul RTO (Retransmission Timeout – temporizare de retransmisie)

Folosind RFC (Request for Comments – document de standardizare IETF) 6298 ($\alpha = 1/8$, $\beta = 1/4$, granularitatea G neglijabilă), calculați SRTT (Smoothed Round-Trip Time – timp dus-întors netezit), RTTVAR (Round-Trip Time Variation – variația timpului dus-întors) și RTO (Retransmission Timeout – temporizare de retransmisie) după eșantioanele RTT (Round-Trip Time – timp dus-întors) de 100 ms și apoi 140 ms.

> **Rezolvare**
>
> Eșantionul 1 (100 ms): SRTT (Smoothed Round-Trip Time – timp dus-întors netezit) = 100, RTTVAR (Round-Trip Time Variation – variația timpului dus-întors) = 50, RTO (Retransmission Timeout – temporizare de retransmisie) = 100 + 4·50 = **300 ms**.
>
> Eșantionul 2 (140 ms): RTTVAR (Round-Trip Time Variation – variația timpului dus-întors) = 0,75·50 + 0,25·|100 − 140| = 37,5 + 10 = 47,5 ms; SRTT (Smoothed Round-Trip Time – timp dus-întors netezit) = 0,875·100 + 0,125·140 = 105 ms; RTO (Retransmission Timeout – temporizare de retransmisie) = 105 + 4·47,5 = **295 ms**.
>
> Observație: RFC (Request for Comments – document de standardizare IETF) 6298 recomandă o limită inferioară de 1 s pentru RTO (Retransmission Timeout – temporizare de retransmisie); Linux folosește un minimum de 200 ms. Cu limita de 1 s, ambele valori ar fi rotunjite în sus la 1 s.

Problema 8 — Decodificarea antetelor dintr-un dump hexazecimal

(a) Decodificați antetul UDP (User Datagram Protocol) `C3 50 00 35 00 2C 1A 2B`. (b) Decodificați primii 16 octeți ai unui antet TCP (Transmission Control Protocol): `D4 31 01 BB 00 00 03 E8 00 00 00 00 A0 02 FA F0`

> **Rezolvare**
>
> (a) Port sursă 0xC350 = **50000**; port destinație 0x0035 = **53 (DNS (Domain Name System – sistemul numelor de domeniu))**; lungime 0x002C = **44 de octeți** → 36 de octeți de date utile; sumă de control 0x1A2B. Este o interogare DNS (Domain Name System – sistemul numelor de domeniu) trimisă de pe un port efemer.
>
> (b) Port sursă 0xD431 = **54321**; port destinație 0x01BB = **443 (HTTPS (Hypertext Transfer Protocol Secure))**; număr de secvență 0x000003E8 = **1000**; număr de confirmare 0 (încă nevalid); deplasament date 0xA = 10 cuvinte = **40 de octeți** (20 de octeți de opțiuni); indicatori 0x02 = **SYN (Synchronize – sincronizare)**; fereastră 0xFAF0 = **64.240 de octeți**. Este primul segment al unei stabiliri de conexiune în trei pași.

Exerciții propuse (temă)

1. Explicați de ce DNS (Domain Name System – sistemul numelor de domeniu) folosește în mod normal UDP (User Datagram Protocol), dar trece la TCP (Transmission Control Protocol) pentru răspunsuri mari și pentru transferul de zone.
2. O legătură prin satelit are R = 50 Mbit/s și RTT (Round-Trip Time – timp dus-întors) = 600 ms; cadrele au 1500 de octeți. Calculați gradul de utilizare la stop-and-wait și dimensiunea ferestrei (în cadre și în octeți) pentru utilizare completă.
3. Un emițător Go-Back-N cu W = 4 trimite cadrele 0–6; cadrul 2 se pierde. Enumerați toate cadrele transmise până la confirmarea cadrului 6. Repetați pentru Selective Repeat.
4. Desenați diagrama de timp a unei încercări de conectare la un port închis. Ce segmente apar? Ce vede aplicația client?

5. Explicați diferența dintre CLOSE_WAIT și TIME_WAIT și precizați în ce stare se află fiecare parte a conexiunii.
6. Pornind de la cwnd = 2 MSS (Maximum Segment Size – dimensiunea maximă a segmentului) și ssthresh = 12 MSS (Maximum Segment Size – dimensiunea maximă a segmentului), urmăriți cwnd pe 15 runde dacă temporizatorul expiră la sfârșitul rundei 8 (aici Tahoe și Reno se comportă la fel).
7. De ce ar putea o aplicație video bazată pe UDP (User Datagram Protocol) să sufere mai puțin la pierderea unui singur pachet decât una bazată pe TCP (Transmission Control Protocol)? Ce ar putea pierde în schimb?

Partea a III-a — Laborator

Laboratorul are două variante echivalente, cu aceleași rezultate ale învățării. Studenții lucrează pe varianta corespunzătoare calculatorului din laborator (sau pe ambele, dacă timpul permite). Fiecare variantă durează aproximativ 100 de minute. Notați observațiile, rezultatele comenzilor (capturi de ecran sau text copiat) și răspunsurile în referatul de laborator (modelul se află la final).

Tabelul 13. *Harta sarcinilor de laborator pentru ambele variante*

Sarcină	Scop	Instrumente Windows	Instrumente Linux
1	Listarea porturilor și conexiunilor deschise	netstat, Get-NetTCPConnection, Get-NetUDPEndpoint	ss, lsof
2	Registrul numerelor de port și intervalul efemer	fișierul services, netsh	/etc/services, sysctl
3	Testarea accesibilității TCP (Transmission Control Protocol)	Test-NetConnection	nc -zv
4	Capturarea stabilirii conexiunii în trei pași	Wireshark (Npcap) sau pktmon	tcpdump, Wireshark
5	Schimb local TCP (Transmission Control Protocol) și UDP (User Datagram Protocol), stări TCP (Transmission Control Protocol)	ncat sau socket-uri .NET din PowerShell	nc (netcat)
6	Setări TCP (Transmission Control Protocol) și controlul congestiei	netsh, Get-NetTCPSetting, netstat -s	sysctl, ss -ti
7	Debitul în prezența întârzierilor și pierderilor	iperf3 (două PCs (Personal Computers – calculatoare personale))	tc netem + iperf3

Siguranță și reguli de conduită

- Scanați, capturați sau testați doar gazdele care vă aparțin sau pe care cadrul didactic le-a desemnat pentru laborator.
- Ștergeți la finalul sesiunii toate regulile temporare de firewall și toate setările de control al traficului.
- Capturile de pachete pot conține parole și date personale: nu le distribuiți în afara cursului.

Laboratorul 6-W: nivelul transport în Windows (Windows 10/11)

Cerințe preliminare

- Windows 10 (versiunea 2004 sau mai nouă) ori Windows 11; Windows PowerShell 5.1 sau PowerShell 7.
- Wireshark cu driverul **Npcap**, instalat cu opțiunea *Support loopback traffic capture* (`winget install WiresharkFoundation.Wireshark`).
- Opțional: **ncat** (instalat împreună cu Nmap, `winget install Insecure.Nmap`) și **iperf3**.
- `curl.exe` este inclus în Windows 10/11. Unele comenzi necesită *Run as administrator* (rulare ca administrator) — acest lucru este indicat.

Sarcina W1 — Listarea porturilor și conexiunilor

Linie de comandă / PowerShell

```
netstat -an                       # toate punctele terminale TCP (Transmission Control Protocol)/UDP (User
Datagram Protocol), numeric
netstat -ano                      # ... plus PID (Process Identifier – identificatorul procesului) al
procesului proprietar
netstat -anob                     # ... plus numele executabilului (administrator)
netstat -an | findstr LISTENING   # doar socket-urile TCP (Transmission Control Protocol) de ascultare
```

PowerShell

```
Get-NetTCPConnection | Sort-Object State |
  Format-Table LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwningProcess -AutoSize

Get-NetTCPConnection -State Listen | Select-Object LocalAddress,LocalPort,OwningProcess
Get-NetUDPEndpoint | Select-Object LocalAddress,LocalPort,OwningProcess
Get-Process -Id (Get-NetTCPConnection -LocalPort 135 -State Listen).OwningProcess
```

Întrebări W1

- Ce porturi TCP (Transmission Control Protocol) sunt în starea LISTEN pe calculatorul vostru? Identificați procesul pentru trei dintre ele.
- De ce punctele terminale UDP (User Datagram Protocol) nu au coloana State?
- Deschideți o pagină web într-un browser și rulați din nou comanda. Ce porturi la distanță apar? Ce porturi locale s-au folosit și în ce interval se află?

Sarcina W2 — Registrul porturilor și intervalul de porturi efemere

PowerShell

```
Get-Content C:\Windows\System32\drivers\etc\services | Select-String "^(ftp|ssh|domain|http|https|ntp)\s"
netsh int ipv4 show dynamicport tcp
netsh int ipv4 show dynamicport udp
```

Întrebări W2

- Ce porturi corespund serviciilor listate? Coincid cu tabelul 4 de la curs?
- Care este intervalul efemer în Windows? Câte conexiuni simultane de ieșire către același port de server permite el de la o singură adresă IP (Internet Protocol)?

Sarcina W3 — Testarea accesibilității TCP (Transmission Control Protocol)

PowerShell

```
Test-NetConnection www.example.com -Port 443        # TcpTestSucceeded : True
Test-NetConnection 127.0.0.1 -Port 12345           # port închis -> False (s-a primit RST (Reset –
resetare))
Test-NetConnection www.example.com -Port 443 -InformationLevel Detailed
```

Un port închis pe o gazdă accesibilă răspunde la SYN (Synchronize – sincronizare) cu **RST (Reset – resetare)**, deci testul eșuează rapid. Un port filtrat de un firewall nu răspunde, iar testul eșuează doar după expirarea unui temporizator. Comparați timpul necesar în cele două cazuri.

Sarcina W4 — Capturarea stabilirii conexiunii în trei pași cu Wireshark

1. Porniți Wireshark, selectați interfața de rețea activă (Ethernet sau Wi-Fi) și începeți captura.

2. Aplicați filtrul de afișare `tcp.port == 80 && ip.addr == <server-address>` — sau simplu `http || tcp.flags.syn == 1`.
3. În PowerShell, generați o conexiune HTTP (Hypertext Transfer Protocol) simplă: `curl.exe -s -o NUL http://example.com` (aflați IP (Internet Protocol)-ul cu `Resolve-DnsName example.com`).
4. Opriți captura. Localizați segmentele **SYN (Synchronize – sincronizare)**, **SYN-ACK (Synchronize-Acknowledgment – sincronizare și confirmare)** și **ACK (Acknowledgment – confirmare)**. În detaliile pachetului, extindeți *Transmission Control Protocol* și notați porturile, numerele de secvență/ACK (Acknowledgment – confirmare), indicatorii, fereastra și opțiunile (MSS (Maximum Segment Size – dimensiunea maximă a segmentului), SACK_PERM, WS (Window Scale – scalarea ferestrei)).
5. Dezactivați numerele de secvență relative (*Edit → Preferences → Protocols → TCP (Transmission Control Protocol) → Relative sequence numbers*) și observați ISNs (Initial Sequence Numbers – numere de secvență inițiale) reale.
6. Localizați segmentele FIN (Finish – terminare) care închid conexiunea. Cine a închis primul?
7. Deschideți *Statistics → Flow Graph* (selectați *TCP (Transmission Control Protocol) Flows*) și salvați diagrama pentru referat.

Fără Wireshark: Packet Monitor inclus în sistem (administrator)

```
pktmon filter remove
pktmon filter add -p 80
pktmon start --capture --pkt-size 0
curl.exe -s -o NUL http://example.com
pktmon stop
pktmon etl2pcap PktMon.etl -o handshake.pcapng
```

Fișierul .pcapng rezultat poate fi deschis ulterior în Wireshark pe orice calculator.

Întrebări W4

- Notați x (ISN (Initial Sequence Number – număr de secvență inițial) al clientului) și y (ISN (Initial Sequence Number – număr de secvență inițial) al serverului). Verificați că ack = x + 1 și ack = y + 1 în segmentele 2 și 3.
- Ce MSS (Maximum Segment Size – dimensiunea maximă a segmentului) anunță fiecare parte? Explicați valoarea pornind de la MTU (Maximum Transmission Unit – unitatea maximă de transmisie).
- Ce factor de scalare a ferestrei se folosește? Care este fereastra de recepție reală în primul segment de date?
- Câte segmente sunt necesare pentru închidere în captura voastră: trei sau patru? De ce ar putea fi trei?

Sarcina W5 — Schimb local TCP (Transmission Control Protocol) și UDP (User Datagram Protocol); stări TCP (Transmission Control Protocol)

Varianta A — cu ncat. Deschideți două ferestre PowerShell.

TCP (Transmission Control Protocol) cu ncat

```
# Fereastra 1 (server)              # Fereastra 2 (client)
ncat -l 5000                     ncat 127.0.0.1 5000
# tastați linii în oricare fereastră; ele apar în cealaltă
```

Observarea stărilor (a treia fereastră)

```
netstat -ano | findstr :5000      # două intrări ESTABLISHED (câte una din fiecare capăt)
# închideți CLIENTUL cu Ctrl+C, apoi imediat:
netstat -ano | findstr :5000      # TIME_WAIT pe partea care a închis prima
```

UDP (User Datagram Protocol) cu ncat

```
# Fereastra 1                        # Fereastra 2
ncat -u -l 6000                  ncat -u 127.0.0.1 6000
Get-NetUDPEndpoint -LocalPort 6000
```

Varianta B — fără instrumente suplimentare (PowerShell + .NET).

Server și client TCP (Transmission Control Protocol) în PowerShell

```
# Fereastra 1: server TCP (Transmission Control Protocol) pentru o singură conexiune, pe portul 5000
$l = [System.Net.Sockets.TcpListener]::new([System.Net.IPAddress]::Loopback, 5000)
$l.Start(); "listening..."
$c = $l.AcceptTcpClient()
$r = [System.IO.StreamReader]::new($c.GetStream()); "received: " + $r.ReadLine()
$c.Close(); $l.Stop()

# Fereastra 2: client
$c = [System.Net.Sockets.TcpClient]::new("127.0.0.1", 5000)
$w = [System.IO.StreamWriter]::new($c.GetStream()); $w.WriteLine("salut nivel transport"); $w.Flush()
$c.Close()
```

Capturați schimbul în Wireshark pe **Adapter for loopback traffic capture**, cu filtrul `tcp.port == 5000 || udp.port == 6000`.

Întrebări W5

- Câte pachete se schimbă pentru a trimite o linie prin TCP (Transmission Control Protocol) (inclusiv stabilirea și închiderea conexiunii)? Dar prin UDP (User Datagram Protocol)?
- Ce se întâmplă dacă trimiteți o datagramă UDP (User Datagram Protocol) către portul 6000 când nu rulează niciun server? Ce arată Wireshark (căutați ICMP (Internet Control Message Protocol))? Ce arată ncat?
- Care parte ajunge în TIME_WAIT? Cât timp rămâne intrarea?

Sarcina W6 — Setări TCP (Transmission Control Protocol) și controlul congestiei

PowerShell

```
netsh int tcp show global
Get-NetTCPSetting | Select-Object SettingName, CongestionProvider, AutoTuningLevelLocal,
InitialCongestionWindowMss
netstat -s -p tcp          # segmente trimise, primite, retransmise, conexiuni resetate
```

Întrebări W6

- Ce algoritm de control al congestiei (CongestionProvider) se folosește pentru șabloanele *Internet* și *Datacenter*?
- Care este fereastra de congestie inițială (în MSS (Maximum Segment Size – dimensiunea maximă a segmentului))? Comparați cu valoarea de la curs.
- Ce este *Receive Window Auto-Tuning* și cum se leagă de opțiunea Window Scale observată în sarcina W4?
- Calculați procentul de retransmisii din `netstat -s` (segmente retransmise / segmente trimise).

Sarcina W7 (opțională) — Debitul între două PCs (Personal Computers – calculatoare personale) din laborator cu iperf3

PowerShell

```
# Pe PC (Personal Computer – calculator personal) 1 (server, administrator: permiteți temporar portul)
New-NetFirewallRule -DisplayName "CN Lab iperf3" -Direction Inbound -Protocol TCP -LocalPort 5201 -Action
Allow -Profile Private
iperf3 -s

# Pe PC (Personal Computer – calculator personal) 2 (client)
iperf3 -c <pc1-address> -t 10            # fereastra implicită
iperf3 -c <pc1-address> -t 10 -w 16K     # fereastră mică: debit limitat de W/RTT (Round-Trip Time – timp
dus-întors)
iperf3 -c <pc1-address> -t 10 -u -b 50M  # UDP (User Datagram Protocol) la 50 Mbit/s: observați pierderile
și jitter-ul

# Curățenie pe PC (Personal Computer – calculator personal) 1
Remove-NetFirewallRule -DisplayName "CN Lab iperf3"
```

Întrebări W7

- Comparați debitul cu fereastra implicită și cu o fereastră de 16 KB (kiloocteți). Folosiți ping pentru a estima RTT (Round-Trip Time – timp dus-întors) și verificați formula Debit ≈ W / RTT (Round-Trip Time – timp dus-întors).
- În modul UDP (User Datagram Protocol), ce raportează iperf3 și nu poate raporta pentru TCP (Transmission Control Protocol)?

Laboratorul 6-L: nivelul transport în Linux (Ubuntu 22.04/24.04 sau Debian 12)

Cerințe preliminare

Instalarea instrumentelor

```
sudo apt update
sudo apt install -y iproute2 tcpdump netcat-openbsd iperf3 curl dnsutils lsof wireshark
```

Comenzile marcate cu `sudo` necesită drepturi de administrator. Sintaxa netcat de mai jos este pentru **netcat-openbsd** (implicit în Ubuntu/Debian); cu netcat *tradițional* folosiți `nc -l -p 5000` în loc de `nc -l 5000`.

Sarcina L1 — Listarea porturilor și conexiunilor

Terminal

```
ss -tuna                 # toate socket-urile TCP (Transmission Control Protocol) (-t) și UDP (User
Datagram Protocol) (-u), numeric (-n), toate stările (-a)
sudo ss -tlnp            # socket-urile TCP (Transmission Control Protocol) de ascultare, cu procesul (-p)
sudo ss -ulnp            # socket-urile UDP (User Datagram Protocol), cu procesul
ss -s                    # statistici sumare
ss -tan state established
sudo lsof -i :22         # ce proces folosește portul 22
```

Întrebări L1

- Ce porturi TCP (Transmission Control Protocol) ascultă pe calculatorul vostru și ce procese le dețin?
- Deschideți o pagină web (`curl -s https://example.com > /dev/null` într-o buclă sau un browser) și listați conexiunile stabilite. Ce interval de porturi locale folosesc?

Sarcina L2 — Registrul porturilor și intervalul efemer

Terminal

```
grep -wE "^(ftp|ssh|domain|http|https|ntp)" /etc/services
getent services ssh
sysctl net.ipv4.ip_local_port_range        # implicit: 32768 60999
sysctl net.ipv4.ip_unprivileged_port_start  # porturile sub această valoare necesită root (implicit 1024)
```

Sarcina L3 — Testarea accesibilității TCP (Transmission Control Protocol)

Terminal

```
nc -zv example.com 443        # "succeeded!" -> conexiunea s-a stabilit
nc -zv 127.0.0.1 12345        # "Connection refused" -> s-a primit RST (Reset – resetare)
nc -zv -w 3 10.255.255.1 80   # niciun răspuns -> expirare (filtrat / inaccesibil)
```

Sarcina L4 — Capturarea stabilirii conexiunii în trei pași cu tcpdump

Terminal

```
# Terminalul 1: captură (-nn numeric, -S numere de secvență absolute)
sudo tcpdump -i any -nn -S 'tcp port 80 and host example.com'
# (rulați din nou cu  -w hs.pcap  pentru a salva pachetele într-un fișier pentru Wireshark)


# Terminalul 2: generați o conexiune
curl -s -o /dev/null http://example.com
```

Rezultat tipic (prescurtat; adresele și numerele vor fi diferite)

```
IP 10.0.2.15.41218 > 93.184.215.14.80: Flags [S], seq 2954731205, win 64240,
   options [mss 1460,sackOK,TS val 1 ecr 0,nop,wscale 7], length 0
IP 93.184.215.14.80 > 10.0.2.15.41218: Flags [S.], seq 1270041366, ack 2954731206, win 65535,
   options [mss 1460,...], length 0
IP 10.0.2.15.41218 > 93.184.215.14.80: Flags [.], ack 1270041367, win 502, length 0
IP 10.0.2.15.41218 > 93.184.215.14.80: Flags [P.], seq 2954731206:2954731281, ack 1270041367, length 75:
HTTP: GET / HTTP/1.1
```

Tabelul 14. *Notația indicatorilor în tcpdump*

Indicator tcpdump	Semnificație
[S]	SYN (Synchronize – sincronizare)
[S.]	SYN (Synchronize – sincronizare) + ACK (Acknowledgment – confirmare)
[.]	Doar ACK (Acknowledgment – confirmare)
[P.]	PSH (Push – livrare imediată) + ACK (Acknowledgment – confirmare) (date)
[F.]	FIN (Finish – terminare) + ACK (Acknowledgment – confirmare)
[R] / [R.]	RST (Reset – resetare) (resetare)

Deschideți `hs.pcap` în Wireshark (`wireshark hs.pcap`) pentru a folosi Flow Graph. Răspundeți la aceleași întrebări ca la **Întrebări W4**.

Sarcina L5 — Schimb local TCP (Transmission Control Protocol) și UDP (User Datagram Protocol); stări TCP (Transmission Control Protocol)

TCP (Transmission Control Protocol) cu nc

```
# Terminalul 1 (server)          # Terminalul 2 (client)
nc -l 5000                    nc 127.0.0.1 5000


# Terminalul 3: observare
ss -tan '( sport = :5000 or dport = :5000 )'
sudo tcpdump -i lo -nn -S 'tcp port 5000'
# închideți clientul (Ctrl+C), apoi:
ss -tan state time-wait '( sport = :5000 or dport = :5000 )'
```

UDP (User Datagram Protocol) cu nc

```
# Terminalul 1                   # Terminalul 2
nc -u -l 6000                 nc -u 127.0.0.1 6000
ss -uan 'sport = :6000'
sudo tcpdump -i lo -nn 'udp port 6000 or icmp'
echo test | nc -u -w1 127.0.0.1 6001     # port UDP (User Datagram Protocol) închis -> ICMP (Internet
Control Message Protocol) port unreachable
```

> **Întrebări L5**
>
> - Răspundeți la **Întrebări W5** pentru Linux.
> - Comparați durata TIME_WAIT în Linux (aproximativ 60 s) cu ce ați observat în Windows.

Sarcina L6 — Fereastra glisantă, cwnd și algoritmii de control al congestiei

Terminal

```
sysctl net.ipv4.tcp_congestion_control            # de obicei: cubic
cat /proc/sys/net/ipv4/tcp_available_congestion_control
sudo modprobe tcp_bbr && cat /proc/sys/net/ipv4/tcp_available_congestion_control
sysctl net.ipv4.tcp_window_scaling net.ipv4.tcp_sack net.ipv4.tcp_ecn
ss -ti dst example.com          # pentru fiecare conexiune: cwnd, ssthresh, rtt, rto, mss, wscale
```

Lăsați să ruleze o descărcare lungă (de exemplu `curl -o /dev/null <address of a large file>`, cum ar fi o imagine de instalare Linux de pe o oglindă indicată de cadrul didactic) și rulați de mai multe ori `ss -ti` pentru a urmări cum se modifică **cwnd**, **rtt** și **rto**.

Sarcina L7 — Debitul în prezența întârzierilor și pierderilor, cu tc netem

Terminal

```
# Terminalul 1
iperf3 -s


# Terminalul 2: măsurătoare de referință pe loopback
iperf3 -c 127.0.0.1 -t 10


# Emulați o WAN (Wide Area Network – rețea de arie largă) pe interfața loopback: 50 ms întârziere în fiecare
sens, 1 % pierderi
sudo tc qdisc add dev lo root netem delay 50ms loss 1%
ping -c 3 127.0.0.1                    # RTT (Round-Trip Time – timp dus-întors) este acum de aproximativ
100 ms
iperf3 -c 127.0.0.1 -t 15 -i 1         # urmăriți coloanele Retr și Cwnd
iperf3 -c 127.0.0.1 -t 15 -i 1 -C reno   # același test cu Reno
iperf3 -c 127.0.0.1 -t 15 -i 1 -C bbr    # același test cu BBR (Bottleneck Bandwidth and Round-trip
propagation time) (modul încărcat la L6)
sudo tc qdisc change dev lo root netem delay 50ms loss 5%


# ÎNTOTDEAUNA ștergeți emularea la final
sudo tc qdisc del dev lo root
```

Atenție

Regula netem afectează **tot** traficul loopback al calculatorului (inclusiv serviciile locale). Nu uitați comanda de curățenie `sudo tc qdisc del dev lo root`.

Întrebări L7

- Completați un tabel: algoritm (cubic, reno, bbr) × pierderi (0 %, 1 %, 5 %) → debit mediu și număr de retransmisii.
- Reprezentați grafic coloana Cwnd pentru un test. Recunoașteți pornirea lentă și reacțiile la pierderi?
- De ce debitul algoritmilor bazați pe pierderi este atât de sensibil la pierderile aleatoare, în timp ce BBR (Bottleneck Bandwidth and Round-trip propagation time) este mai puțin afectat?

Model de referat de laborator

Tabelul 15. *Structura referatului de laborator*

Secțiune	Conținut
1. Identificare	Nume, grupă, dată, variantă (Windows / Linux), versiunea OS (Operating System – sistem de operare)
2. Porturi și conexiuni	Rezultatele sarcinilor 1–2, cu serviciile și procesele identificate
3. Analiza stabilirii conexiunii	Captură de ecran/flow graph; tabel cu seq/ack/indicatori pentru stabilire și închidere; MSS (Maximum Segment Size – dimensiunea maximă a segmentului) și scalarea ferestrei
4. Experimentul TCP (Transmission Control Protocol) comparativ cu UDP (User Datagram Protocol)	Numărul de pachete, observarea TIME_WAIT, comportamentul porturilor UDP (User Datagram Protocol)/TCP (Transmission Control Protocol) închise
5. Controlul congestiei	Algoritmul folosit, cwnd inițial, măsurătorile din sarcina 7, cu o scurtă interpretare
6. Concluzii	Trei lucruri învățate și o întrebare care rămâne deschisă

Bibliografie

- J. F. Kurose, K. W. Ross — *Computer Networking: A Top-Down Approach*, ed. a 8-a, Pearson, 2021 — capitolul 3.
- A. S. Tanenbaum, N. Feamster, D. Wetherall — *Computer Networks*, ed. a 6-a, Pearson, 2021 — capitolul 6.
- RFC (Request for Comments – document de standardizare IETF) 768 (UDP (User Datagram Protocol)), RFC (Request for Comments – document de standardizare IETF) 9293 (TCP (Transmission Control Protocol)), RFC (Request for Comments – document de standardizare IETF) 6298 (RTO (Retransmission Timeout – temporizare de retransmisie)), RFC (Request for Comments – document de standardizare IETF) 5681 (controlul congestiei TCP (Transmission Control Protocol)), RFC (Request for Comments – document de standardizare IETF) 7323 (scalarea ferestrei, marcaje de timp), RFC (Request for Comments – document de standardizare IETF) 9438 (CUBIC), RFC (Request for Comments – document de standardizare IETF) 6335 (registrul porturilor).
- Wireshark User's Guide — wireshark.org/docs; pagini de manual: ss(8), tcpdump(8), nc(1), tc-netem(8); Microsoft Learn: Get-NetTCPConnection, Test-NetConnection, pktmon.